Sialkot AQI Forecaster

End-to-End MLOps Air Quality Prediction System

Submitted to: 10Pearls
Prepared by: Haroon Waqar
June 2026
Live Dashboard: https://aqi-prediction-sialkot.streamlit.app/

1. Executive Summary

The Sialkot AQI Forecaster is a production-grade, fully serverless MLOps pipeline that autonomously predicts PM2.5 concentrations and converts them to the US EPA Air Quality Index (AQI) scale for Sialkot, Pakistan, 72 hours in advance.

Unlike a static notebook, the system continuously ingests live data, retrains its model daily, and serves forecasts via a real-time web dashboard all without manual intervention and at zero infrastructure cost.

Component	Technology	Status
Feature Store	Hopsworks Serverless	Live
Model Registry	Hopsworks Model Registry	Live
CI/CD Pipeline	GitHub Actions	Live
Web Dashboard	Streamlit Community Cloud	Live
Weather Data	Open-Meteo API	Live
Pollutant Data	OpenWeatherMap API	Live

2. Project Overview

2.1 Objective

To build an autonomous, end-to-end air quality forecasting system for Sialkot that:

- Ingests live hourly weather and pollutant data from external APIs
- Stores engineered features in a cloud feature store
- Trains and registers the best ML model on a daily schedule
- Serves 72-hour PM2.5 forecasts via a public web dashboard
- Converts raw PM2.5 values to the official US EPA 0-500 AQI scale with health advisories

2.2 Target Variable

The model predicts continuous PM2.5 concentration (micrograms per cubic metre, μg/m³) as a regression problem. PM2.5 was chosen over the raw 1-5 AQI level because:

- PM2.5 is a real physical measurement that ML models can predict with numerical precision
- The 1-5 AQI level is a coarse classification with wide, lossy buckets
- The official US EPA formula can convert any PM2.5 value to a precise 0-500 AQI score, giving users more informative output

3. System Architecture

The system is decoupled into three independent pipelines, each with a distinct responsibility and automation schedule. This separation ensures that a failure in one pipeline does not cascade into others.

Pipeline	File	Trigger	Responsibility
Feature Pipeline	feature_pipeline.py	Every hour (GitHub Actions)	Fetch, merge, engineer, and store features in Hopsworks
Training Pipeline	training_pipeline.py	Daily 00:45 UTC (GitHub Actions)	Train champion model and push to Hopsworks Model Registry
Inference Pipeline	app.py	On user request (Streamlit Cloud)	Serve 72-hour PM2.5 + AQI forecast via web dashboard

4. Feature Pipeline

4.1 Data Sources

Two APIs were combined to construct each training row:

- Open-Meteo (free, no API key): wind_speed_100m, precipitation, apparent_temperature, wind_gusts_10m, vapour_pressure_deficit, relative_humidity_2m
- OpenWeatherMap Air Pollution API: co, no2, o3, so2, pm2_5, pm10, nh3, aqi (1-5 level)

Both sources return hourly data and are merged on the datetime column using an inner join.

4.2 Dynamic Time Window

Rather than hardcoding dates, the pipeline dynamically computes a 2-day lookback window on every run. This ensures no hours are ever missed between GitHub Actions executions, and Hopsworks deduplicates on (city, datetime) so re-ingested overlapping rows are harmless.

4.3 Feature Engineering

Feature	Source	Engineering Logic
wind_speed_100m	Open-Meteo	Raw
precipitation	Open-Meteo	Raw
apparent_temperature	Open-Meteo	Raw
wind_gusts_10m	Open-Meteo	Raw
vapour_pressure_deficit	Open-Meteo	Raw

Feature	Source	Engineering Logic
relative_humidity_2m	Open-Meteo	Raw
co, no2, o3, so2, nh3	OpenWeather	Raw pollutant concentrations
temp_lag1	Derived	apparent_temperature.shift(1)
wind_rolling_6	Derived	wind_speed_100m.rolling(6).mean()
day_of_week	Derived	datetime.dt.dayofweek (0=Mon)
month	Derived	datetime.dt.month
is_weekend	Derived	Binary flag: 1 if Sat/Sun
hour_sin	Derived	sin(2π x hour / 24)
hour_cos	Derived	cos(2π x hour / 24)

Note: PM2.5 lags were deliberately excluded. Using pm2_5_lag1 would be valid for 1-step-ahead forecasting but is invalid for 72-hour direct forecasting (the lag value is unknown in the future). Weather lags are safe because Open-Meteo provides a forecast API.

5. Training Pipeline

5.1 Data Split Strategy

A critical decision was the choice of train/test split. The initial implementation used scikit-learn's train_test_split with random shuffling, which produced misleadingly high metrics. This was replaced with a strict chronological split (shuffle=False) to simulate real production conditions.

Split Method	R^2 (XGBoost)	Trustworthy?	Reason
Random (shuffle=True)	0.847	No	Temporal leakage - model sees future data during training
Chronological (shuffle=False)	0.634	Yes	Model only ever sees past data; future test set is genuinely unseen

5.2 Model Bake-off Results

Model	RMSE (µg/m³)	MAE (µg/m³)	R^2	Selected?
Ridge Regression	19.76	15.12	0.078	No
Random Forest	14.89	11.39	0.477	No
XGBoost Regressor	15.57	11.96	0.634	Yes

XGBoost was selected as the champion model due to its highest R^2 and ability to capture non-linear interactions between atmospheric variables (temperature inversions, humidity spikes, wind shear) that are characteristic of Sialkot's smog patterns.

5.3 Champion Model Configuration

XGBRegressor(learning_rate=0.1, max_depth=6, n_estimators=200, random_state=42)

6. Inference Pipeline (Web App)

6.1 Dashboard Features

- 72-hour PM2.5 line chart — continuous hourly forecast
- 3-day AQI summary cards — date, label, emoji, daily average PM2.5
- Dynamic health alerts: Green (AQI < 101), Yellow (101-150), Red (≥ 151)
- Raw hourly data table with Predicted_PM25 and Predicted_AQI columns
- Streamlit caching: model cached with @st.cache_resource, weather data with @st.cache_data(ttl=3600)

6.2 US EPA AQI Conversion

Each PM2.5 prediction is converted to the 0-500 AQI scale using the official EPA linear interpolation formula: AQI = ((I_high - I_low) / (C_high - C_low)) x (C - C_low) + I_low

PM2.5 (µg/m³)	AQI Range	Category
0.0 - 12.0	0 - 50	Good
12.1 - 35.4	51 - 100	Moderate
35.5 - 55.4	101 - 150	Unhealthy for Sensitive Groups
55.5 - 150.4	151 - 200	Unhealthy
150.5 - 250.4	201 - 300	Very Unhealthy
250.5 - 500.4	301 - 500	Hazardous

7. CI/CD Automation

Two GitHub Actions workflows run the pipelines on schedule without any manual intervention:

Workflow	Cron Schedule	Purpose
feature_pipeline.yml	0 * * * * (every hour)	Fetch latest weather and pollutant data, push to Hopsworks Feature Store
training_pipeline.yml	45 0 * * * (daily 00:45 UTC)	Retrain XGBoost on fresh data, push updated model to Model Registry

Daily retraining ensures the model continuously adapts to concept drift as Sialkot's air quality conditions evolve seasonally.

8. Exploratory Data Analysis

EDA was performed on the Feb-Jun 2026 Sialkot hourly dataset (approximately 2,880 rows). Key findings:

- Seasonal trend: PM2.5 is significantly elevated in February-March (winter smog season driven by temperature inversions and agricultural burning), declining toward June as thermal mixing improves.
- Daily cycle: PM2.5 peaks twice daily around 08:00-10:00 (morning commute + inversion) and 20:00-22:00 (evening cooling). This justified cyclical sine/cosine hour encoding.
- Wind speed: Strong negative correlation with PM2.5 (~-0.62). High wind disperses pollutants rapidly, making it the single most predictive meteorological feature.
- Humidity: Positive correlation with PM2.5 at humidity > 80%, consistent with hygroscopic particle growth.
- Pollutant co-occurrence: CO and NO2 showed the highest raw correlation with PM2.5 (~0.73 and ~0.61), reflecting shared combustion emission sources.
- Weekend effect: Marginally lower morning PM2.5 on weekends, likely reflecting reduced industrial activity on Friday-Saturday in Pakistan.

9. Blockades Faced and Resolutions

The following technical blockades were encountered during development. Each required investigation and a deliberate architectural decision to resolve.

Blockade 1: Random Train/Test Split Causing Data Leakage

Problem: The initial pipeline used scikit-learn's train_test_split with random shuffling. This produced an R^2 of 0.947 which appeared excellent. Upon investigation, the model was training on data from March and May while testing on February effectively seeing the future during training. The metric was completely dishonest.

Resolution: Replaced random split with a chronological 80/20 split using shuffle=False after sorting the dataframe by datetime. R^2 dropped to 0.622 (Random Forest) and 0.634 (XGBoost), a real and honest reflection of model performance on future data.

Blockade 2: PM2.5 Lag Features Breaking at 72-Hour Inference

Problem: Adding pm2_5_lag1 and pm2_5_lag24 as features dramatically improved training R^2 (up to 0.70+). However, these features are unknown at inference time for a 72-hour direct forecast. Using them would require a recursive forecasting loop where each prediction becomes the next input, compounding errors across all 72 steps.

Resolution: Decided to exclude all PM2.5 lags from the feature set. Only lags on meteorological variables (temp_lag1, wind_rolling_6) were kept, since those are available from Open-Meteo's

forecast API. This sacrifices some accuracy for a prediction system that actually works correctly at inference time.

Blockade 3: Streamlit Cloud Server Timezone Offset

Problem: The Streamlit Community Cloud runs on UTC. Pakistan Standard Time is UTC+5. Without correction, the 'filter to next 3 days' logic was using UTC midnight as 'today', causing the dashboard to show the wrong 3-day window for Pakistani users sometimes including data from the previous local day, sometimes skipping one.

Resolution: Imported pytz and anchored all date comparisons to Asia/Karachi timezone: today = datetime.now(pytz.timezone('Asia/Karachi')).date(). This ensures the dashboard always shows the correct next 3 local days regardless of which server Streamlit assigns.

Blockade 4: API Horizon Mismatch Between Open-Meteo and OpenWeatherMap

Problem: Open-Meteo provides forecast data to midnight of Day 5 (exactly 120 hours from the request). OpenWeatherMap's Air Pollution Forecast provides data from the exact request timestamp, meaning the trailing evening hours of Day 5 exist in the weather dataframe but are missing in the pollutant dataframe. An inner merge deleted those rows.

Resolution: Switched from an inner merge to a left merge, with Open-Meteo as the authoritative datetime spine. Applied .ffill() to forward-fill the last available pollutant values into the trailing hours where OpenWeatherMap data was missing. This preserves the full forecast window without deleting valid weather rows.

10. Limitations and Future Work

10.1 Current Limitations

- No PM2.5 lags: Deliberate architectural decision for true 72-hour direct forecasting, but limits R^2 compared to recursive 1-step-ahead systems.
- 4-month training window: Sialkot's severe winter smog (November-January, AQI frequently > 300) is not in the training data. Predictions in that season will be less reliable until more data accumulates.
- Single city: The pipeline is parameterised for Sialkot only. Multi-city support would require schema changes and city-specific model training.
- No deep learning baseline: An LSTM was not included in the initial bake-off.

10.2 Planned Improvements

- Extend training data to November-January to capture Sialkot's worst smog season
- Add LSTM model to the bake-off for time-series-native comparison
- Multi-city support with city selector dropdown in the UI
- Historical accuracy tracking: compare yesterday's prediction vs. actual measured AQI

11. Conclusion

The Sialkot AQI Forecaster successfully delivers on all core requirements of the project brief: a feature pipeline, a training pipeline, automated CI/CD, a Feature Store and Model Registry, a live web dashboard, health alerts, and official US EPA AQI conversion all deployed on a 100% serverless, zero-cost infrastructure stack.

The most significant technical achievement is the architectural integrity of the system: a chronological data split that prevents leakage, feature engineering that is consistent between training and inference, and production edge-case handling (timezone correction, inference padding, API mismatch) that would otherwise silently corrupt predictions in a deployed system.

The model's R^2 of 0.634 on genuinely unseen future data is an honest result for a 72-hour direct PM2.5 forecast without PM2.5 lags, and will improve organically as the hourly pipeline accumulates more training data across seasons.

Tech Stack Summary

Layer	Tool/Service	Purpose
Machine Learning	XGBoost, scikit-learn, pandas, numpy, SHAP	Model training and feature engineering
Feature Store	Hopsworks Serverless	Versioned feature storage and retrieval
Model Registry	Hopsworks Model Registry	Serialized model storage with metrics
Automation	GitHub Actions	Hourly feature ingestion, daily retraining
Web App	Streamlit Community Cloud	Public forecast dashboard
Weather API	Open-Meteo	Historical + forecast meteorological data
Pollutant API	OpenWeatherMap	Historical + forecast pollutant concentrations
Serialization	joblib (.pkl)	Model save and load
Timezone	pytz	Pakistan Standard Time correction on UTC server